Sim2Real Robotics Hackathon - Simulation Round

Participant README: setup, tasks, scoring and submission rules

1. Overview

Sim2Real is a two-stage autonomous robotics competition:

1. **Simulation Round (this repository):** Contestants develop, benchmark, and validate their autonomy stack inside a Gazebo Harmonic digital-twin environment across three progressive difficulty tiers.
2. **Hardware Round:** Qualifying teams deploy their software directly onto the physical differential-drive robot platform inside the real-world arena.

In the **Simulation Round**, contestants program the robot inside a 3.0 x 3.0 m walled digital-twin arena. The objective is to autonomously navigate to procedurally generated waypoints, dwell at each target for at least 2.00 s, avoid obstacles, and return home to the start zone.

2. Quickstart

System Prerequisites

Ensure ROS 2 Jazzy, Gazebo Harmonic, and bridging packages are installed:

```
sudo apt update && sudo apt install -y \
  ros-jazzy-ros-gz \
  ros-jazzy-cv-bridge \
  ros-jazzy-tf2-ros \
  python3-opencv \
  python3-numpy
```

Understanding the ROS 2 Workspace and Directory Rules

A ROS 2 workspace is the top-level directory containing the `src/` directory. In this repository, the workspace root is `postman_ws/`:

- **ALWAYS run `colcon build` from the workspace root** (the top-level folder where `src/` and `README.md` reside).
- **NEVER run `colcon build` from inside `src/` or inside a package folder** (for example inside `src/postman_pkg` or `src/postman_solution`). Doing so will create nested, broken build directories.

Step-by-Step Build Instructions

1. Navigate to the workspace root directory. Open a terminal and `cd` into the directory containing `src/`:

```
cd ~/postman_ws   # Or the folder path where you cloned/extracted this repository
```

Verify your location: running `ls` should list `src/` and `README.md`.

2. Source the ROS 2 underlay. Load the core ROS 2 Jazzy environment:

```
source /opt/ros/jazzy/setup.bash
```

3. Build the packages. Execute the build command from the workspace root:

```
colcon build --symlink-install
```

This generates `build/`, `install/`, and `log/` directories at the workspace root alongside `src/`.

4. Source the workspace overlay. Register the newly built packages in your active terminal:

```
source install/setup.bash
```

> **IMPORTANT: Multi-Terminal Sourcing** Testing your code requires running the simulator, your solution, and the evaluator in **separate terminal windows**. Every time you open a new terminal, you must source both ROS 2 Jazzy and the workspace before running any commands:
>
> ```
> source /opt/ros/jazzy/setup.bash
> source install/setup.bash
> ```
>
> Tip: adding `source /opt/ros/jazzy/setup.bash` to your `~/.bashrc` ensures ROS 2 is automatically loaded in every new terminal.

Running the Competition Stack

Terminal 1: Launch the Simulation Environment

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
# Launch Tier 1, 2, or 3 (seed can be changed to test different layouts)
ros2 launch postman_pkg sim.launch.py tier:=1 seed:=42
```

Terminal 2: Run the Automated Evaluator

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
# Monitor and score your run
python3 src/postman_pkg/scripts/evaluator.py --tier 1
```

Terminal 3: Launch Your Solution

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
# Run your autonomous solution
ros2 launch postman_solution solution.launch.py tier:=1
```

3. Repository Structure and Files to Work On

The workspace contains two packages:

```
postman_ws/
├── src/
|   ├── postman_pkg/              # [ORGANIZER PACKAGE - DO NOT MODIFY]
|   |   ├── config/               # Bridge config, RViz layouts
|   |   ├── launch/sim.launch.py  # Simulation launch file
|   |   ├── postman_pkg/          # Procedural waypoint spawner, noise
injection
|   |   ├── scripts/evaluator.py  # Automated evaluation harness
|   |   ├── urdf/robot.urdf       # Robot physical and kinematic description
|   |   └── worlds/               # Gazebo Harmonic world descriptions
|   |
|   └── postman_solution/         # [CONTESTANT PACKAGE - WORK HERE]
|       ├── launch/
|       |   └── solution.launch.py
|       └── postman_solution/
|           ├── tier1_node.py     # Tier 1 Solution Node
|           ├── tier2_node.py     # Tier 2 Solution Node
|           ├── tier3_node.py     # Tier 3 Solution Node
|           └── solution_node.py  # Default / Generic Entry Point
└── README.md
```

Contestant Boundary

- **Work exclusively in `src/postman_solution`.** All your code, nodes, helper modules, and launch logic must reside here.
- **Do NOT modify `src/postman_pkg`.** During official evaluation, your `postman_solution` package is evaluated in a clean organizer environment with an unmodified `postman_pkg`. Any reliance on modified simulation files will result in evaluation failure.

4. Robot Physical Specifications and Arena Invariants

All measurements are in SI units (meters, radians, seconds).

Parameter	Value	Description
Wheel Radius	`0.0225 m` (22.5 mm)	Radius of driven traction wheels
Wheel Separation	`0.159 m` (159 mm)	Center-to-center track gauge between drive wheels

Parameter	Value	Description
Max Linear Velocity	`0.30 m/s`	Hard forward velocity limit
Min Linear Velocity	`0.00 m/s`	**Reverse motion is strictly prohibited** (v >= 0)
Max Angular Velocity	`1.50 rad/s`	Symmetrical yaw rate limit (w in [-1.5, 1.5] rad/s)
Robot Mass	`0.82 kg` (0.70 kg chassis + wheels/caster)	Total assembly mass
Footprint Envelope	`X in [-0.010, 0.200] m, Y in [-0.0885, 0.0885] m`	Bounding envelope in `base_link` frame
Start / Home Zone	`0.30 x 0.30 m`	Red square centered at (0.0, 0.0) on the arena floor
Waypoint Geometry	`0.15 m` (radius)	Green cylinder disks flush with the arena floor
Dwell Requirement	`>= 2.00 s`	Minimum continuous stopping duration (|v| <= 0.03 m/s)
Overhead Camera	`4.00 m` height	Positioned at (1.50, 1.50, 4.00) m, pointing directly down
Camera Sensor	`800 x 800` px, 70 degree FOV	Covers the entire arena floor
Arena Dimensions	`3.0 x 3.0 m`	Enclosed by four perimeter boundary walls

5. Tasks to be Done by Tier

Tier 1: Dead Reckoning Navigation

File to implement: `src/postman_solution/postman_solution/tier1_node.py`

In Tier 1, target waypoint coordinates are published directly on ROS topics.

Tasks:

1. **State Estimation**

- Subscribe to `/joint_states` (wheel encoder velocities, noisy) and `/imu` (gyroscope and accelerometer).
- Estimate the robot pose (x, y, theta) in the `odom` frame.
- Publish `nav_msgs/msg/Odometry` on `/odom`.
- Broadcast the dynamic transform from frame `odom` to frame `base_link` using TF2.

2. **Point-to-Point Navigation**
   - Subscribe to `/waypoints` (`geometry_msgs/msg/PoseArray`), which contains the initial 5 target positions.
   - Steer the robot using `/cmd_vel` to visit each target.
3. **Waypoint Dwell Verification**
   - At each target, bring the robot to a complete stop (|v| <= 0.03 m/s) within the waypoint capture radius for at least 2.00 s continuously.
4. **Dynamic Waypoint Handling**
   - Subscribe to `/dynamic_waypoints` (`geometry_msgs/msg/PoseArray`).
   - At T+5.0 s after the robot first moves, 2 additional dynamic waypoints are published.
   - Navigate to and collect these dynamic waypoints as well (total 7 waypoints).
5. **Return Home**
   - Return to the red start zone at (0.0, 0.0) and come to a complete stop for at least 2.00 s to conclude the run.

Tier 2: Perception-Guided Navigation

File to implement: `src/postman_solution/postman_solution/tier2_node.py`

In Tier 2, coordinate topics `/waypoints` and `/dynamic_waypoints` are **suppressed**. Target locations are not provided via ROS messages and must be detected visually.

Tasks:

1. **Camera Stream Ingestion**
   - Subscribe to `/overhead_camera/image_raw` (`sensor_msgs/msg/Image`) and `/overhead_camera/camera_info` (`sensor_msgs/msg/CameraInfo`).
2. **Visual Waypoint Detection**
   - Detect the green waypoint disks visible on the arena floor.
   - Determine their coordinates in the arena coordinate frame.

3. **Dynamic Waypoint Detection**
   - At T+7.0 s after the robot first moves, 2 dynamic green waypoint disks visually spawn onto the arena floor.
   - Detect and register these new targets mid-run.
4. **Autonomous Navigation and Dwell**
   - Navigate to all 7 waypoints using your estimated odometry.
   - Fulfill the >= 2.00 s continuous dwell requirement at each target.
5. **Return Home**
   - Return to the (0.0, 0.0) red start zone and stop for >= 2.00 s.

Tier 3: Cluttered Arena and Obstacle Avoidance

File to implement: `src/postman_solution/postman_solution/tier3_node.py`

In Tier 3, coordinate topics remain suppressed, and the arena contains 4 wall obstacles with **magenta tops**.

Tasks:

1. **Obstacle and Waypoint Perception**
   - Detect both the green waypoint disks and the magenta-topped obstacle walls from `/overhead_camera/image_raw`.
   - Determine the spatial layout of all obstacles and targets.
2. **Collision-Free Path Planning**
   - Plan navigation routes that visit all waypoints while strictly avoiding collisions with:
     - The 4 interior obstacle walls.
     - The 4 outer perimeter boundary walls.
3. **Dynamic Waypoint Integration**
   - Detect the 2 dynamic waypoints that visually appear at T+7.0 s and update your navigation path.
4. **Waypoint Collection and Dwell**
   - Collect all 7 waypoints with the >= 2.00 s dwell constraint at each.
5. **Return Home with Zero Collisions**
   - Return to the (0.0, 0.0) start zone without touching any obstacle or wall.

6. Automated Evaluation and Scoring Protocol

Runs are officially evaluated using `src/postman_pkg/scripts/evaluator.py`, which subscribes to ground-truth simulation data:

```
python3 src/postman_pkg/scripts/evaluator.py --tier <1|2|3>
```

Evaluation Criteria

1. **Waypoints Collected (N / 7)**
   - Robot must enter the waypoint capture radius (0.20 m center-to-center).
   - Robot must come to a continuous stop (|v| <= 0.03 m/s) for **at least 2.00 s**.
2. **Collisions (Tier 3 and Perimeter)**
   - Contact between the robot chassis bounding box and any perimeter wall or obstacle wall registers a collision infraction.
3. **Return to Home**
   - Robot must return to within 0.20 m of (0.0, 0.0) and remain stopped for >= 2.00 s.
4. **Run Duration (Sim Time)**
   - Clock starts automatically on the first `/cmd_vel` movement command and stops when the robot completes the home return dwell.
5. **Total Odometry Distance**
   - Ground-truth trajectory length traveled by the robot.

7. ROS 2 Interface Matrix

Topics Available to Contestants

Topic	Type	Rate	Description
`/joint_states`	`sensor_msgs/msg/JointState`	100 Hz	Wheel velocities with injected noise (sigma = 0.002 rad/s)
`/imu`	`sensor_msgs/msg/Imu`	100 Hz	Gyroscope yaw rate (`angular_velocity.z`) and linear accelerations

Topic	Type	Rate	Description
`/overhead_camera/image_raw`	`sensor_msgs/msg/Image`	30 Hz	800 x 800 RGB overhead arena image stream
`/overhead_camera/camera_info`	`sensor_msgs/msg/CameraInfo`	30 Hz	Camera intrinsics
`/waypoints`	`geometry_msgs/msg/PoseArray`	Latched	*(Tier 1 only)* Initial 5 static target poses in `odom` frame
`/dynamic_waypoints`	`geometry_msgs/msg/PoseArray`	Latched	*(Tier 1 only)* 2 dynamic target poses emitted at T+5.0 s
`/clock`	`rosgraph_msgs/msg/Clock`	~1000 Hz	Simulation clock (`use_sim_time: True`)

Topics Expected from Contestants

Topic	Type	Requirement	Description
`/cmd_vel`	`geometry_msgs/msg/Twist`	**Mandatory**	Commanded linear (v in [0.0, 0.3] m/s) and angular (w in [-1.5, 1.5] rad/s) velocity
`/odom`	`nav_msgs/msg/Odometry`	**Mandatory**	Your estimated robot pose and velocity in `odom` frame
TF Transform	`tf2_ros` Broadcaster	**Mandatory**	Broadcast transform `odom` to `base_link` for RViz visualization

8. Submission Rules and Integrity Constraints

1. **Submission Launch Command:** Your package will be launched and evaluated using:

   ```
   ros2 launch postman_solution solution.launch.py tier:=<TIER_NUMBER>
   ```

2. **Hidden Evaluation Seeds:** Submissions will be tested against undisclosed random seeds:

```
ros2 launch postman_pkg sim.launch.py tier:=<TIER_NUMBER> seed:=
<EVAL_SEED>
```

Hardcoded coordinates will fail. Your solution must dynamically handle procedurally generated layouts.

3. **Cheating Disqualification:**
   - Subscribing to `/evaluator/*` ground-truth topics or internal `/gz/*` topics is strictly forbidden and results in immediate disqualification.
   - All navigation and perception must rely strictly on the public contestant topics listed in Section 7.